



Факультет Программной Инженерии и Компьютерной техники

Программирование

Лабораторная работа №3-4

Вариант: Фунчоза

Выполнил: Малых Кирилл Романович

Группа: Р3132

Преподаватель: Разинкин Александр Владимирович

Санкт-Петербург, 2025г.

Содержание

1. Условие поставленной задачи	3
2. Требования к объектной модели, сценарию и программе:	4
3. Текст (изображение) исходного задания	5
4. Составленный на основе картинки концептуальное взаимодействие объектов моей предметной области:	6
5. Сравнительный анализ метода, написанной мною и сгенерированный искусственным интеллектом	8
6. Исходные файлы для лабораторной работы	9
7. Заключение.....	10
8. Список использованной литературы	11

1. Условие поставленной задачи

- 1) Получить вариант
- 2) Нарисовать UML-диаграмму, представляющую классы и интерфейсы объектной модели и их взаимосвязи;
- 3) Придумать сценарий, содержащий действия персонажей, аналогичные приведенным в исходном тексте;
- 4) Согласовать диаграмму классов и сценарий с преподавателем.
- 5) Написать программу на языке Java, реализующую разработанные объектную модель и сценарий взаимодействия и изменения состояния объектов. При запуске программа должна проигрывать сценарий и выводить в стандартный вывод текст, отражающий изменение состояния объектов, приблизительно напоминающий исходный текст полученного отрывка.
- 6) Для одного из методов класса персонажа (или объекта), согласованного с преподавателем, создать промпты для 2-3 ИИ-ассистентов (Gigachat, DeepSeek, Qwen, или др.), получить от них код метода, реализовать метод самостоятельно, и включить в отчет сравнительный анализ кода, написанного ИИ-ассистентами, и своего варианта. Оценить качество кода, полученного от ИИ-ассистентов, указать на возможные недоработки.
- 7) Продемонстрировать выполнение программы на сервере helios.

2. Требования к объектной модели, сценарию и программе:

В модели должны быть представлены основные персонажи и предметы, описанные в исходном тексте. Они должны иметь необходимые атрибуты и характеристики (состояние) и уметь выполнять свойственные им действия (поведение), а также должны образовывать корректную иерархию наследования классов.

Объектная модель должна реализовывать основные принципы ООП - инкапсуляцию, наследование и полиморфизм. Модель должна соответствовать принципам SOLID, быть расширяемой без глобального изменения структуры модели.

Сценарий должен быть вариативным, то есть при изменении начальных характеристик персонажей, предметов или окружающей среды, их действия могут изменяться и отклоняться от базового сценария, приведенного в исходном тексте. Кроме того, сценарий должен поддерживать элементы случайности (при генерации персонажей, при задании исходного состояния, при выполнении методов).

Объектная модель должна содержать как минимум один корректно использованный элемент каждого типа из списка:

абстрактный класс как минимум с одним абстрактным методом;

интерфейс;

перечисление (enum);

запись (record);

массив или ArrayList для хранения однотипных объектов;

проверяемое исключение.

В созданных классах основных персонажей и предметов должны быть корректно переопределены методы `equals()`, `hashCode()` и `toString()`. Для классов-исключений необходимо переопределить метод `getMessage()`.

Созданные в программе классы-исключения должны быть использованы и обработаны. Кроме того, должно быть использовано и обработано хотя бы одно `unchecked` исключение (можно свое, можно из стандартной библиотеки).

При необходимости можно добавить внутренние, локальные и анонимные классы.

3. Текст (изображение) исходного задания



4. Составленный на основе картинки концептуальное взаимодействие объектов моей предметной области:

Игровое пространство представляет из себя прямоугольное футбольное поле размером 20 на 12. В момент запуска программы основной игры, создаются объекты креветок и одной улитки, а также акул. Их местоположение определяется случайно и задаётся 2-мя координатами в рамках футбольного поля. Мяч случайным образом даётся одной из креветок. Основной метод у креветок и улиток – сделать пас. Возможность сделать пас определяется по следующему критерию: если расстояние до ближайшего игрока меньше 4 единиц (перебираем всевозможных игроков на поле), то пасуем ему. Пас подаётся самому ближайшему игроку, входящего в диапазон 4 единиц.

С увеличением времени продолжительности матча, расстояние до ближайшего игрока увеличивается. Если никому сделать пас не получается, то через некоторое время координаты креветок и улиток меняются. Гол можно забить лишь только в том случае, если игрок с мячом находится в радиусе 7 единиц от точки ворот. Удар по воротам осуществляется с шансом, который зависит от коэффициента уверенности улитки или креветки и расстояния до ворот, что указывается в полях объекта.

Главные противники креветок и улитки – акулы. Их особенность заключается в том, что они с шансом 15% могут съесть игрока вашей команды, если попадают в радиус 3 единиц к какому-нибудь из игроков (попросту удаляется объект), при этом с увеличением времени продолжительности матча, этот шанс может увеличиваться. Если в момент поедания у игрока был мяч, или это была улитка фунчоза, то игра считается проигрышной для вас, так как мяч проткнули или соответственно ГГ убивают.

Основная суть геймплея акул заключается в том, что если при изменении их координат они попали в радиус 5 единицы от мяча (ну или текущего игрока, у

которого есть мяч), то используя метод ударом плавником она с 20% может попасть в ворота (независимо от расстояния). Но также есть 20%, что мяч может далеко улететь мимо ворот, что закончится поражением для команды акул. Оставшиеся 60% можем считать попаданием в штангу, мяч передаётся обратно той креветке, у которой отобрали мяч. Также есть дополнительный коэффициент, который зависит от расстояния от точки местонахождения акулы и самих ворот. Чем ближе акула, тем больше этот коэффициент, соответственно, шанс на попадание в сами ворота больше. Акула делает удар на основе $2 * \text{коэффициент уверенности}$.

После гола раунд как бы начинается сначала, мяч снова передаётся любой из креветок.

Конец игры считается тогда, когда одна из команд забивает 5 голов.

Стандартное количество креветок вместе с улиткой: 7

Количество акул: 3

Улитка имеет уникальные статы, например, после смены местоположения с шансом 20% улитка использует метод shout, чтобы поднять коэффициент уверенности всей команды на 30%.

По ходу каждого действия должно выводиться сообщение о совершённом действии, например, при успешном использовании паса должно выводиться, что пасующий передал мяч тому-то и так далее.

5. Сравнительный анализ метода, написанной мною и сгенерированный искусственным интеллектом

Для нашего анализа возьмём в качестве примера метод `showStartupAnimation()` класса `StartupAnimation`. Составим подходящий промпт для нейросетей, который подробно объяснял идейную реализацию выбранного метода:

“Создай метод `showStartupAnimation()` на Java 17 для класса `StartupAnimation`, опираясь на следующие условия:

1. Воспроизвести звук при помощи метода `playSound()` для объекта `intro`.
2. Очистить консоль при помощи метода `clearConsole` реализованного в классе интерфейса `clearConsoleSpace`
3. Построчно вывести логотип "Юнона геймс" с помощью метода `GetStudioHeader()` класса `introHeaders` с задержкой 130 мс между строками
4. После полного вывода ждать 2300 мс
5. Очистить консоль и вывести заголовок "Фунчоза", используя метод `GetFunchozaHeader()` класса `introHeaders`
6. Через 4000 мс очистить консоль
7. Обработать `InterruptedException` с `Thread.currentThread().interrupt()` Используй `try/catch` и корректное управление исключениями”

В результате данного запроса DeepSeek написал следующий код:

<https://github.com/tetraminomusic/ITMO/blob/main/Программирование/Лабораторные%20работы/ЛБ34/DeepSeekPrompt.java>

В результате данного запроса GigaChat следующий код:

<https://github.com/tetraminomusic/ITMO/blob/main/Программирование/Лабораторные%20работы/ЛБ34/GigaChat.java>

Анализируя свой метод и метод, сгенерированные другими нейросетями, можно заметить, что в моём варианте вся реализация данного метода хранится внутри лишь тела этого метода, в то время как ИИ-шные коды разделяют на несколько под-методов. В рамках данного простого метода это не имеет смысла, так как функционал не такой уж и обширный, чтобы вводить это. Если DeepSeek просто разделил на несколько простых методов, то GigaChat перемудрил с кодом: добавил debug-режим. Для больших методов, где требуется большой объём данных, такая реализация была к месту, например, если бы заставка была адаптированной под определённый размер окна.

6. Исходные файлы для лабораторной работы

UML-диаграммы доступны по следующей ссылке:

<https://github.com/tetraminomusic/ITMO/tree/main/Программирование/Лабораторные%20работы/ЛБ34>

“*uml_new.png*” – UML-диаграмма итогового проекта;

“*uml_old.png*” – Первоначальная UML-диаграмма (прототип);

Исходный код программы доступен по следующей ссылке:

https://github.com/tetraminomusic/funchoza_source_code

7. Заключение

В ходе данной лабораторной работы я впервые оказался на месте гейм-дизайнера, которому потребовалось с нуля придумать игровые механики – как объекты тех или иных классов могли бы взаимодействовать друг с другом. Помимо всего прочего, я чуть глубже ознакомился с ООП и обработками ошибок в Java, что явно будет полезно в будущем при написании следующих лабораторных работах. Помимо всего прочего, я научился более-менее оформлять пользовательский интерфейс, работать с переходами между определёнными “фреймами” и так далее.

8. Список использованной литературы

- 1) Хидео Кодзима – “Хидео Кодзима. Гены гения” – 320 с.
- 2) Роберт Мартин – “Чистая архитектура. Искусство разработки программного обеспечения” – 352 с.
- 3) Грейди Буч, Джеймс Рамбо, Айвар Якобсон – “Язык UML. Руководство пользователя” – 496 с.
- 4) Мартин Фаулер – “Рефакторинг. Улучшение существующего кода” – 448 с.
- 5) Алан Шаллоуей, Джеймс Р. Тротт – “Паттерны проектирования. Новый подход к объектно-ориентированному анализу и проектированию” – 288 с.